Introduction to Machine Learning

Lab 2: Supervised Learning

Comparing Classifiers on the Breast Cancer Dataset

Student:	Your Name
Student ID:	Student ID
Instructor:	Zhang Li & Jing Ming
Institution:	Shandong Women's University
Date:	2026.10.21

Based on `notebooks/02-supervised-learning-code.ipynb`. Replace all placeholder text before submission.

Contents

1 Objectives **2**

2 Background **2**

3 Dataset **2**

4 Methodology **3**

5 Experiments and Results **3**
5.1 Exercise 1 — Data split 3
5.2 Exercise 2 — Logistic regression 3
5.3 Exercise 3 — Decision tree and overfitting 3
5.4 Exercise 4 — Random forest and feature importances 4
5.5 Exercise 5 — Scaling matters for SVMs 5
5.6 Exercise 6 — Probability estimates 5

6 Discussion **5**

7 Conclusion **5**

A Source Code **5**

1 Objectives

In this lab you compare several supervised classifiers on the same dataset. By the end you should be able to:

- Train several classifiers and compare **train vs. test** accuracy.
- Observe **overfitting** in an unrestricted decision tree.
- Read **feature importances** from a random forest.
- Explain why **SVMs need feature scaling**.
- Obtain class **probability estimates** with `predict_proba`.

2 Background

In *supervised learning* a model is trained on labelled examples $(\mathbf{x}_i, y_i)$ and learns a mapping $f : \mathbf{x} \mapsto y$ that should **generalise** to unseen data. Here $y \in \{0, 1\}$ (malignant / benign), so this is a **binary classification** task. We report accuracy on a held-out test set,

$$\text{accuracy} = \frac{\#\text{ correct predictions}}{\#\text{ samples}},$$

and compare it to the training accuracy to detect over-/under-fitting:

- **Overfitting** — high training accuracy but a noticeably lower test accuracy (the model memorised training quirks).
- **Underfitting** — low accuracy on both sets (the model is too simple).

This lab touches five model families: **logistic regression**, **decision trees**, **random forests**, **support vector machines (SVM)**, and uses **feature scaling** (`StandardScaler`) and **probability estimates** (`predict_proba`).

3 Dataset

We use the **Breast Cancer Wisconsin** dataset, bundled with scikit-learn as `load_breast_cancer()`. Each sample describes a tumour with 30 numeric features; the target labels it *malignant* or *benign*.

Table 1: Dataset summary (fixed facts, provided for reference).

Property	**Value**
Source	`sklearn.datasets.load_breast_cancer`
Task type	Binary classification
Number of samples	569
Number of features	30 (numeric)
Classes	malignant (0), benign (1)
Split	`train_test_split(..., random_state=0)`
Training samples	426
Test samples	143

4 Methodology

The notebook is organised as six exercises. All models come from scikit-learn and share the same train/test split (`random_state=0`). The setup:

Listing 1: Imports and data split (Exercise 1).

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, random_state=0)   # -> 426 train / 143
        test
```

The six exercises are:

1. Load the data and create the train/test split.
2. Train `LogisticRegression(max_iter=5000)`; report train/test accuracy.
3. Train a `DecisionTreeClassifier` with no depth limit and with `max_depth=4`; compare train vs. test to see overfitting.
4. Train `RandomForestClassifier(n_estimators=100, random_state=0)`; report test accuracy and the 5 most important features.
5. Train `SVC()` on raw vs. `StandardScaler`-scaled data.
6. Print `predict_proba` for the first 5 test samples.

5 Experiments and Results

5.1 Exercise 1 — Data split

Report the sizes produced by the split.

- Training samples: *TODO* (expected 426)
- Test samples: *TODO* (expected 143)

5.2 Exercise 2 — Logistic regression

Table 2: Logistic regression accuracy.

Model	Train acc.	Test acc.
`LogisticRegression(max_iter=5000)`	*TODO*	*TODO*

5.3 Exercise 3 — Decision tree and overfitting

Compare an unrestricted tree with a depth-limited one.

Table 3: Decision tree: effect of `max_depth` on over-fitting.

Model	Train acc.	Test acc.	Comment
`DecisionTree(random_state=0)`	*TODO*	*TODO*	overfits?
`DecisionTree(max_depth=4)`	*TODO*	*TODO*	

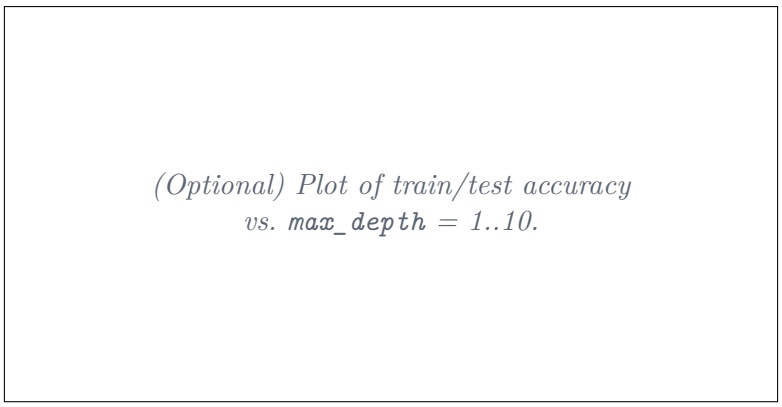


Figure 1: Train/test accuracy as the tree depth increases (optional extension).

5.4 Exercise 4 — Random forest and feature importances

Report the random-forest test accuracy and list the five most important features.

Table 4: Top-5 features by random-forest importance.

Rank	Feature name	Importance
1	*TODO*	*TODO*
2	*TODO*	*TODO*
3	*TODO*	*TODO*
4	*TODO*	*TODO*
5	*TODO*	*TODO*

Random-forest test accuracy: *TODO* (expected > 0.90).

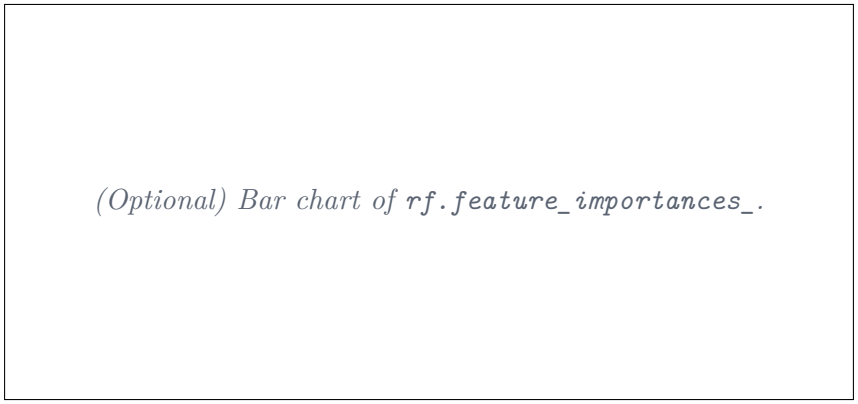


Figure 2: Random-forest feature importances (optional).

5.5 Exercise 5 — Scaling matters for SVMs

Compare the SVM on raw features versus `StandardScaler`-scaled features.

Table 5: SVM test accuracy with and without feature scaling.

Preprocessing	Test acc.
`SVC()` on raw data	*TODO*
`SVC()` on scaled data	*TODO*

5.6 Exercise 6 — Probability estimates

Report `predict_proba` for the first five test samples.

Table 6: Predicted class probabilities for the first 5 test samples.

Sample	P(malignant)	P(benign)	Predicted
1	*TODO*	*TODO*	*TODO*
2	*TODO*	*TODO*	*TODO*
3	*TODO*	*TODO*	*TODO*
4	*TODO*	*TODO*	*TODO*
5	*TODO*	*TODO*	*TODO*

6 Discussion

Answer with evidence from your tables and figures above:

- **Overfitting (Ex. 3).** How large is the gap between train and test accuracy for the unrestricted tree? Does `max_depth=4` reduce it?
- **Best model.** Which classifier generalised best on the test set, and why do you think so?
- **Scaling (Ex. 5).** How much did `StandardScaler` change the SVM's test accuracy? Why are SVMs sensitive to feature scales while trees and forests are not?
- **Feature importances (Ex. 4).** Do the top features in Table 4 make intuitive sense for tumour diagnosis?
- **Probabilities (Ex. 6).** Are any of the five samples predicted with low confidence? How could a probability threshold be useful in a medical setting?

7 Conclusion

A Source Code

Full code: `notebooks/02-supervised-learning-code.ipynb`. Key solution:

Listing 2: Reference solution for Exercises 2–6.

```
# Ex2 - logistic regression
logreg = LogisticRegression(max_iter=5000).fit(X_train, y_train)
print("logreg train/test:", logreg.score(X_train, y_train),
      logreg.score(X_test, y_test))
```

```
5
6  # Ex3 - decision tree (overfitting)
7  full   = DecisionTreeClassifier(random_state=0).fit(X_train, y_train)
8  pruned = DecisionTreeClassifier(max_depth=4, random_state=0).fit(
       X_train, y_train)
9
10 #Optional: Train/test accuracy as the tree depth increases (optional
       extension).  Plot of train/test accuracy vs. max_depth = 1..10
11
12 import matplotlib.pyplot as plt
13
14 plt.figure(figsize=(8, 5))
15 train_acc = []
16 test_acc = []
17 for depth in range(1, 11):
18     tree = DecisionTreeClassifier(max_depth=depth, random_state=0).fit
          (X_train, y_train)
19     train_acc.append(tree.score(X_train, y_train))
20     test_acc.append(tree.score(X_test, y_test))
21 plt.plot(range(1, 11), train_acc, label='Train Accuracy', marker='o')
22 plt.plot(range(1, 11), test_acc, label='Test Accuracy', marker='o')
23 plt.xlabel('Max Depth')
24 plt.ylabel('Accuracy')
25 plt.title('Decision Tree Accuracy vs Max Depth')
26 plt.xticks(range(1, 11))
27 plt.legend()
28 plt.grid()
29 plt.show()
30
31 # Ex4 - random forest + feature importances
32 rf = RandomForestClassifier(n_estimators=100, random_state=0).fit(
       X_train, y_train)
33 top = np.argsort(rf.feature_importances_)[::-1][:5]
34 print(rf.score(X_test, y_test), list(cancer.feature_names[top]))
35
36 # Optional. Random-forest feature importances (optional).  Bar chart
       of rf.feature_importances_
37
38 plt.figure(figsize=(10, 6))
39 importances = rf.feature_importances_
40 indices = np.argsort(importances)[::-1]
41 plt.title('Feature Importances from Random Forest')
42 plt.bar(range(X_train.shape[1]), importances[indices], align='center')
43 plt.xticks(range(X_train.shape[1]), cancer.feature_names[indices],
       rotation=90)
44 plt.tight_layout()
45 plt.show()
46
47 # Ex5 - scaling matters for SVM
48 scaler = StandardScaler().fit(X_train)
49 svc_raw    = SVC().fit(X_train, y_train)
50 svc_scaled = SVC().fit(scaler.transform(X_train), y_train)
51
52 # Ex6 - probability estimates
53 print(np.round(rf.predict_proba(X_test[:5]), 3))
```